IoT Lab Report: Wearable Monitoring for Retirement Homes

Ruben Lohberg*, Guglielmo Cattaneo*, Pascal Petzoldt*
*Karlsruhe Institute of Technology, DE
*{uiurg, ufucw, udsyh}@student.kit.edu

***Abstract*—This paper presents a proof-of-concept monitoring system for elderly care consisting of a wearable sensing node, a wireless receiving station, and a web-based dashboard. The prototype uses the Texas Instruments (TI) CC1350 SensorTag and LaunchPad hardware to collect ambient temperature, room humidity, and motion data, derive a fall alarm from IMU measurements, and transmit the resulting state as JSON packets over a sub-1 GHz RF link. On the PC side, received packets are forwarded via UART, processed by a Python backend, and displayed by a React frontend that can monitor multiple device IDs in one interface. The prototype is not a final product, but it is used to demonstrate the end-to-end information flow, alarm handling, and simplified medication reminder logic of a possible wearable monitoring system.**

I. Introduction

Falls, medication schedules, and room conditions such as temperature and humidity are practical concerns in elderly care. In a retirement home, care staff already manage these tasks, but they must often keep track of several residents at the same time. A wearable monitoring device can support this work by collecting simple status information centrally and by making important alarms visible at a nurse station.

This work presents a proof-of-concept system for such a setup. The prototype demonstrates the basic idea of a patient-side sensing unit, wireless transmission to a station, and a user interface through which care staff can observe data and handle alarms. In a possible product, the wearable would be worn by the patient and integrated into a larger monitoring system. In that setting, skin temperature would be measured by dedicated sensors, and medication reminders would be tied to patient-specific schedules. The prototype simplifies these aspects by using ambient temperature measurements and a fixed reminder interval.

The implemented prototype uses the hardware provided in the lab, namely a Texas Instruments CC1350 SensorTag [1] as the wearable-side node and a CC1350 LaunchPad [2] as the receiving station. This platform is used for prototyping and is not meant to represent a final product design. The battery-powered SensorTag already integrates the required sensors and can communicate with the LaunchPad over RF. The received data are forwarded via UART to a computer, because a PC-based station is closer to the intended use of a central monitoring interface than the LaunchPad's onboard display. In this setup, several SensorTag-LaunchPad pairs can be connected to the same PC, allowing the prototype to display data from multiple residents in one interface. Within the prototype, the measured humidity corresponds to room air humidity, while the measured temperature is ambient temperature at the device and serves only as a stand-in for the skin temperature that a dedicated wearable could provide.

II. Functional Overview

The intended monitoring system can be understood as a sequence of information transformations. Sensor readings and button events are collected on the wearable, condensed into one application packet, transmitted to a receiving station, and then forwarded to a central software system where the information is stored and displayed. This viewpoint emphasizes what information is produced, how it changes, and how it is used, without tying the concept to one specific hardware realization.

A. Sensor Data Gathering

Before transmission, the wearable gathers the information that is later shown in the monitoring interface. This includes temperature and humidity measurements, button events used for acknowledgments, and a unique device identifier that allows the backend to associate each packet with the correct resident. In addition to these measured values, the wearable derives a fall-related acceleration value and an alarm state from motion data. Button information is represented as acknowledgment events rather than continuously maintained states. This keeps each packet self-contained while still allowing short user interactions to be observed at the monitoring station.

B. Fall Detection and Alarm Generation

The fall detection logic transforms three acceleration components into one scalar value. For each sample, the acceleration magnitude a is calculated in units of g as

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2}. \tag{1}$$

This value is used as a compact representation of motion intensity independent of direction. During one transmission interval, the maximum observed acceleration magnitude is retained and later written into the JSON field `a`. If the measured magnitude exceeds the configured threshold, the alarm state becomes active. The alarm remains set until it is acknowledged via button 1.

TABLE I
FIELDS CONTAINED IN ONE TRANSMITTED JSON PACKET.

Field	Meaning
`id`	Unique wearable identifier as a 16-character hexadecimal IEEE address.
`temp`	Temperature value in degree Celsius. In the prototype, this is ambient temperature measured at the device.
`hum`	Relative room air humidity in percent.
`b1`	Indicates a recent button 1 press. In the application, this also acknowledges and clears the fall alarm.
`b2`	Indicates a recent button 2 press for the medication reminder workflow.
`a`	Maximum acceleration magnitude in g observed since the previous packet.
`alarm`	Current fall alarm state. Once set, it remains active until acknowledged.

C. JSON Packet Format

Each transmitted RF message is assembled as one compact JSON object with a fixed key order. The payload combines measurement values, button events, the wearable identifier, and the current alarm state in a single packet. This keeps the interface between the wearable side and the PC side simple, since every transmission already contains the information needed for display and alarm handling. In the current implementation, one JSON object is sent every 500 ms, floating-point values are encoded with two decimal digits, and boolean values are represented as JSON literals.

Table I summarizes the contents of one transmitted packet.

D. RF Communication

At the RF link level, the JSON payload is embedded into a variable-length packet. The transmitter prepends a length byte and a two-byte sequence number to the application message and sends one such packet during each transmission interval. The sequence number is not needed by the web interface itself, but it is useful at the communication layer because it allows individual transmissions to be distinguished during testing. The receiver listens continuously on the same radio settings and extracts the application payload from each packet accepted by the underlying radio configuration. In this way, the RF link presents the higher software layers with one complete JSON message per successful transmission.

E. PC-side Processing and Web Interface

After a packet has been received, the receiving station forwards the decoded JSON message to the PC via UART as one text line. A connector process reads the incoming serial data, parses each line as JSON, and forwards the result to the backend through an ingest endpoint. The backend timestamps each reading, stores the latest state and a short history for every device ID, and additionally maintains the current status of the medication reminder. The frontend periodically queries this backend and presents both an overview of multiple residents and a detail view for one selected resident.

This creates a clear separation of responsibilities. The backend is the stateful part of the PC-side system: it receives sensor packets, keeps per-resident data, implements the reminder logic, and exposes the relevant information through an API. The frontend is kept as a thin interaction layer. It does not maintain its own logic for alarms or reminders, but only requests data from the backend, displays the returned state, and sends the few PC-side user actions, such as medication confirmations, back to the API.

III. SYSTEM OVERVIEW

The proof-of-concept system is built around a TI CC1350 SensorTag, a CC1350 LaunchPad receiver, and a PC-based monitoring application. This section describes how the prototype realizes the system functions through concrete hardware and software components, and where a future product could differ. Figure 1 shows the main components and their connections.

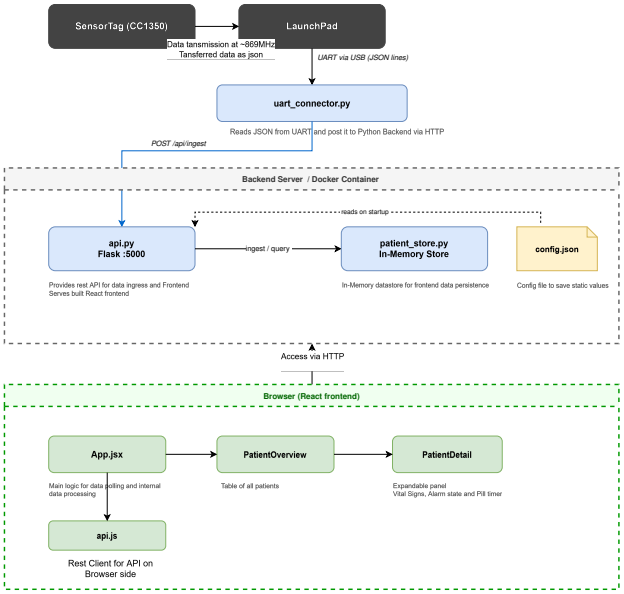

Fig. 1. Architecture of the prototype, showing the SensorTag, LaunchPad, UART connector, backend, and frontend

A. SensorTag Platform and Sensors

The prototype uses the sensors available on the CC1350 SensorTag. In the current firmware, humidity is obtained from the HDC1000, while the transmitted temperature value is formed as the average of the BMP280 and HDC1000 temperature readings. Motion data for fall detection comes from the MPU9250 IMU, whose accelerometer values are used directly in the alarm logic. Gyroscope values are read together with the acceleration data but are not used in the implemented fall decision rule.

In our tests, the SensorTag sensors showed noticeable offsets from reference measurements. For example, the HDC1000 temperature reading differed by about 4 K from a DS18B20 sensor used for comparison. This is acceptable for the proof-of-concept system, but it also shows that the prototype measurements should not be interpreted as medical-grade values.

B. Prototype Fall Detection Implementation

In the prototype, the fall detection logic runs directly on the SensorTag and processes accelerometer data from the onboard IMU. The firmware computes the acceleration magnitude for every sampled IMU reading and stores the maximum value observed since the previous packet transmission. The decision rule uses a fixed empirical threshold chosen so that the alarm behavior is easy to reproduce during testing. A suitable threshold depends on how the device is attached to the body and would need empirical adjustment. Once this threshold is exceeded, the fall alarm is latched and remains active until it is cleared by button 1.

C. Wearable Identification

Each wearable is identified by its IEEE address. During initialization, the firmware first checks whether a custom address is configured in the CCFG area. If no custom value is present, it falls back to the factory-programmed address stored on the device. The selected address is then converted into a 16-character uppercase hexadecimal string and inserted into the JSON field `id`. On the PC side, this identifier is used as the primary key for storing readings, distinguishing residents, and keeping alarms and history data associated with the correct wearable.

D. RF Communication Settings

The transmitter and receiver share one RF configuration generated with TI SmartRF Studio 2.29.0 [3]. In the current prototype, the radio operates in TI proprietary sub-1 GHz mode at a center frequency around 870 MHz with a symbol rate of 50 kBaud. The generated settings use a 32-bit sync word and variable-length packets; whitening is disabled. These parameters are defined in shared SmartRF configuration files and reused by both firmware projects, which reduces the risk of mismatched settings between sender and receiver. The embedded firmware was developed in Code Composer Studio (CCS) [4] and built against the SimpleLink CC13x0 SDK version 4.20.02.07 [5].

E. Communication Protocol and Reliability

The current prototype uses a simple periodic uplink protocol. Each SensorTag transmits autonomously every 500 ms, and the paired LaunchPad receiver listens continuously on the configured channel. This keeps the communication structure simple, but it also means that there is no handshake and no end-to-end retransmission if a packet is missed on the RF link.

This limitation is especially relevant for short acknowledgment actions triggered by the two buttons. If a button press were reported in only one packet, a single packet loss could hide that action completely from the PC side. For this reason, the firmware latches a button press and reports it again in several consecutive packets. This does not remove packet loss, but it makes the acknowledgment event much more likely to arrive at least once. The trade-off is that one physical button press can appear multiple times in the packet stream, which is acceptable here because the relevant actions are acknowledgments rather than continuously varying measurements.

For a future product, the same monitoring function should use synchronous communication and could also be implemented with other wireless protocols such as BLE, LoRaWAN, or Wi-Fi. The monitoring concept is therefore not tied to the proprietary sub-1 GHz RF link used in the prototype.

F. LaunchPad LCD and Debugging Output

Although the LaunchPad mainly serves as a receiver and UART bridge, its onboard LCD was useful during development. The display shows a ready message, the number of received packets, and the most recently decoded application message, wrapped across several lines if necessary. This provides a direct way to verify RF reception and packet contents locally, even before the PC software is running or when UART forwarding is being debugged. The same decoded message is also written to UART, so the LCD and serial output expose the same information path at different points in the prototype.

G. Integration of Multiple Sensor Tags

Two basic approaches were considered for scaling the prototype to several residents. One option is a single receiver shared by several wearables. This would reduce the amount of receiver hardware, but it would also require coordination between the transmitters and explicit handling of packet collisions on the shared radio link. The alternative, used in the prototype, is to pair each SensorTag with its own LaunchPad receiver and to connect several such pairs to one PC. This increases the number of boards and serial connections, but it keeps the radio links independent and makes development, testing, and fault isolation considerably simpler. For the proof-of-concept system, this was the more practical trade-off. For the scalability of a finished product, a shared receiving infrastructure for many wearables would be preferable, but that would require a communication protocol designed explicitly for many concurrent devices.

H. PC-side Software and Development Setup

The PC-side software was designed so that the same backend can be exercised either with real hardware or with simulated data. For hardware-based operation, a UART connector reads one JSON message per line from a LaunchPad serial port and posts it to the backend. For software-only testing, the same connector offers a mock mode that generates plausible packets for one or more device IDs by drawing synthetic temperature, humidity, and acceleration values from predefined ranges. This allows frontend and backend work to continue even when the embedded hardware is not available.

For deployment, the web application is packaged with Docker [6]. This allows a standard deployment process and abstracts away the need for build tools and software frameworks such as Python or npm/vite. The whole web application can be started with a single Docker command. This builds the Docker container with all dependencies and runs it on port 5000.

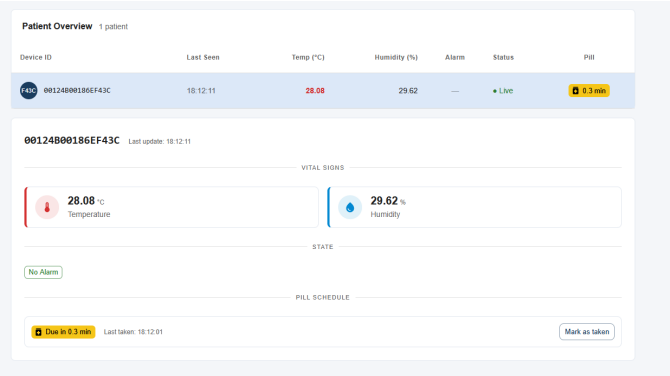

Fig. 2. Screenshot of the web interface, showing live data from SensorTag

1) Backend Implementation: Python with the Flask [7] library is mainly used for serving the backend part of the application. It is designed as a REST API [8]. The web server provides multiple API endpoints such as *GET /api/patients*, which returns a list of available patients with their master data (*id*, *last-seen*) and the latest measurements from the sensor tag. Using a REST API instead of a prerendered webpage supports separation of concerns and supports the integration into other software systems which might be planned in future, for example if the retirement home wanted to extend the system with mobile pager alerts.

2) Frontend Implementation: The frontend is implemented with React [9] as a single-page application (SPA). The browser sends HTTP requests to the backend and displays the returned data in the predefined layout. Data from the backend is refreshed automatically every two seconds, allowing the frontend to stay up to date with the current patient data. Figure 2 shows a screenshot of the frontend, displaying live data gathered from a connected sensor tag.

I. Placement of the Pill Timer

The medication reminder could in principle be implemented on the wearable, on the receiving station, or on the PC side. Placing it on the wearable would couple reminder schedules directly to the embedded firmware and make later changes more cumbersome. Placing it on the receiver would still distribute reminder logic over several hardware devices and would not integrate naturally with the user interface. In the prototype, the reminder is therefore handled in the backend, where the interval can be configured centrally and the resulting status can be exposed directly to the web interface. This also aligns better with a future system in which reminder schedules would be maintained per resident in a central application.

J. Reference Values for Skin Temperature and Room Humidity

In a future wearable, skin temperature would be measured by a dedicated contact temperature sensor, for example a thermistor or RTD, rather than by the ambient-temperature sensors used in the prototype. A single absolute skin-temperature threshold is difficult to define, so an older-adult reference value is more naturally taken from clinical fever guidance: the Infectious Diseases Society of America lists repeated oral temperatures above 37.2 °C or a rise of more than 1.1 °C above baseline as relevant fever criteria in long-term care [10]. A real skin-worn device should therefore use such values only as a medical reference and base its actual warning on a sustained rise above the patient's personal baseline at the chosen wearing position.

For room humidity, a dedicated capacitive humidity sensor would be a plausible choice in a real product. The measured quantity is already directly relevant to the resident's environment, and the Umweltbundesamt recommends a relative humidity range of about 30–60 % for living spaces [11]. A practical warning threshold for the monitoring system can therefore be based on values that remain outside this range for longer periods.

References

[1] Texas Instruments, "CC1350STK: SimpleLink multi-standard sensor tag," https://www.ti.com/tool/CC1350STK, accessed: Mar. 15, 2026.
[2] Texas Instruments, "LAUNCHXL-CC1350: SimpleLink dual-band wireless MCU LaunchPad development kit," https://www.ti.com/tool/LAUNCHXL-CC1350, accessed: Mar. 15, 2026.
[3] Texas Instruments, "SmartRF Studio 7 application software," https://www.ti.com/tool/download/SMARTRF-STUDIO-7/2.29.0, accessed: Mar. 15, 2026.
[4] Texas Instruments, "Code Composer Studio integrated development environment," https://www.ti.com/tool/download/CCSTUDIO, accessed: Mar. 15, 2026.
[5] Texas Instruments, "SimpleLink CC13x0 SDK," https://www.ti.com/tool/SIMPLELINK-CC13X0-SDK, accessed: Mar. 15, 2026.
[6] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, Mar. 2014.
[7] Pallets Projects, "Flask: A Python microframework," https://flask.palletsprojects.com/, accessed: Mar. 15, 2026.
[8] R. T. Fielding and R. N. Taylor, "Principled design of the modern web architecture," *ACM Trans. Internet Technol.*, vol. 2, no. 2, p. 115–150, May 2002. [Online]. Available: https://doi.org/10.1145/514183.514185
[9] Meta Platforms, Inc, "React documentation," https://react.dev/, accessed: Mar. 15, 2026.
[10] "Clinical practice guideline for the evaluation of fever and infection in older adult residents of long-term care facilities: 2008 update by the infectious diseases society of america," https://pmc.ncbi.nlm.nih.gov/articles/PMC7166905/, pMCID: PMC7166905. Accessed: Mar. 15, 2026.
[11] Umweltbundesamt, "Anforderungen an lüftungskonzeptionen in gebäuden - wohngebäude," https://www.umweltbundesamt.de/sites/default/files/medien/1410/publikationen/fb_anforderungen_an_lueftungskonzeptionen_in_wohngebaeuden_bf.pdf, accessed: Mar. 15, 2026.